

AWS VPC Architecture Documentation

This document explains step-by-step creation of a Production-style VPC including Public and Private subnets, Internet Gateway, NAT Gateway, Route Tables, Elastic IP and EC2 instances.

Creating a Custom VPC

I created a custom VPC to design a production-style network architecture instead of using the default VPC.

Configuration:

1.VPC Name: vpc-prod-style

2.CIDR Block: 10.0.0.0/16

A VPC (Virtual Private Cloud) is a logically isolated network in AWS where we launch our resources like EC2 instances.

It allows us to:

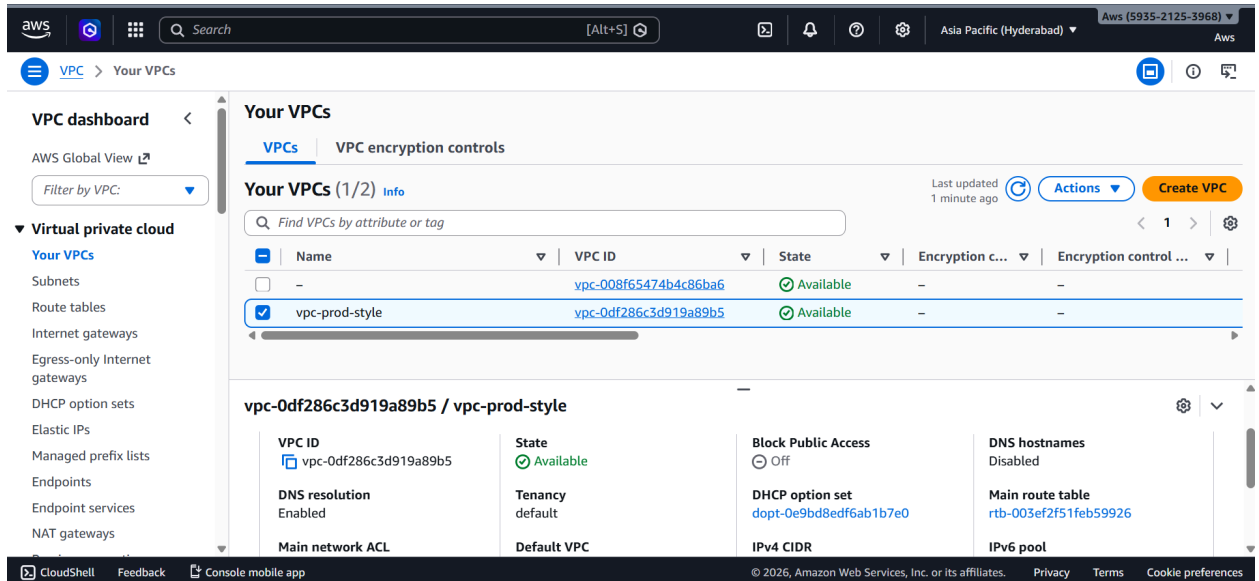
- Define our own IP address range
- Create subnets
- Configure route tables
- Control internet access
- Apply network-level security

Why Do We Need a Custom VPC?

- ✓ To design structured network architecture
- ✓ To separate public and private resources
- ✓ To control traffic flow using route tables
- ✓ To implement production-level security
- ✓ To avoid dependency on default VPC setup

What Happens If We Use a Default VPC?

- Limited architectural control
- Poor network segmentation
- Not suitable for real-world production environments
- Hard to demonstrate real DevOps network design



Creating Public & Private Subnets

After creating the VPC, I divided it into **two subnets** to implement proper network segmentation.

Configuration:

Public Subnet

CIDR: 10.0.1.0/24

Used for internet-facing resources

Private Subnet

CIDR: 10.0.2.0/24

Used for backend/internal resources

What is a Subnet?

A subnet is a subdivision of a VPC's IP range.

It allows us to logically separate resources inside the same VPC.

Why Do We Separate Public & Private Subnets?

Public Subnet:

- ✓ Hosts web servers / bastion hosts
- ✓ Can connect directly to the Internet (via IGW)

Private Subnet:

- ✓ Hosts databases / backend services
- ✓ No direct internet access
- ✓ More secure

What Happens If Everything Is on a Public Subnet?

Databases exposed to internet
Increased attack surface
Poor security architecture
Not production-ready

The screenshot shows the AWS Management Console interface for the 'Subnets' page. The top navigation bar includes the AWS logo, a search bar, and the region 'Asia Pacific (Hyderabad)'. The left sidebar shows the 'VPC dashboard' with a list of VPC resources. The main content area displays a table of subnets with columns for Name, Subnet ID, State, VPC, and Block Public IP. All subnets are in an 'Available' state. Below the table, there is a 'Select a subnet' section.

Name	Subnet ID	State	VPC	Block Public IP
public-subnet-1	subnet-0108b104a29210627	Available	vpc-0df286c3d919a89b5 vpc-...	Off
private-subnet-1	subnet-0a7225cc524b4c82d	Available	vpc-0df286c3d919a89b5 vpc-...	Off
-	subnet-0851fd0261736c04e	Available	vpc-008f65474b4c86ba6	Off
-	subnet-081acd68fd3fd3383	Available	vpc-008f65474b4c86ba6	Off
-	subnet-0c384efd264f81c6	Available	vpc-008f65474b4c86ba6	Off

Attaching an Internet Gateway (IGW)

After creating subnets, I attached an **Internet Gateway (IGW)** to enable internet connectivity for public resources.

What is an Internet Gateway?

An Internet Gateway is a VPC component that allows communication between:

After creating the VPC and subnets, the next step was to enable internet connectivity.

Resources inside the VPC

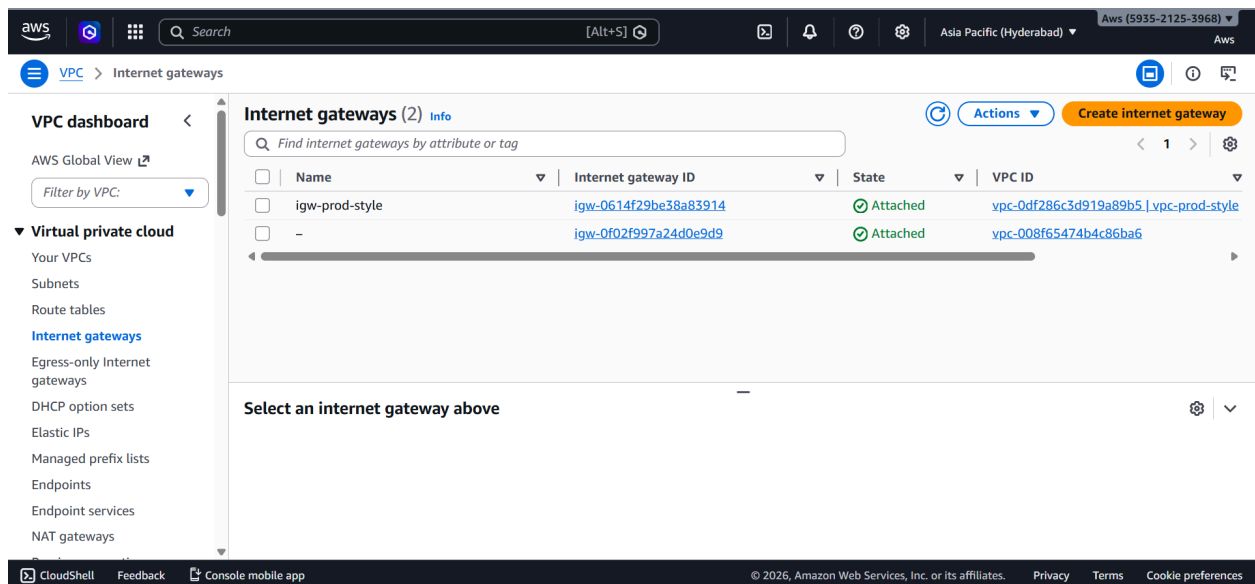
The Public Internet

It acts as a bridge between your VPC and the outside world.

Configuration Done:

- ✓ Created Internet Gateway
- ✓ Attached it to the VPC
- ✓ Added route in Public Route Table

1.Created a new IGW from the VPC console.

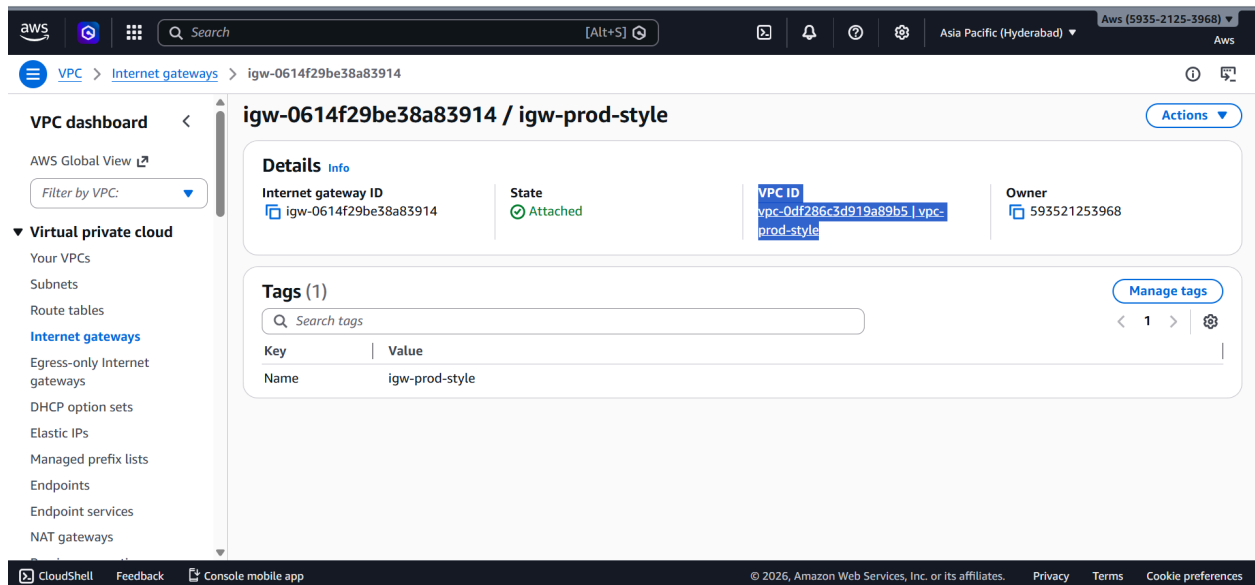


2. Attached IGW to My VPC

After creating it, I attached the IGW to my custom VPC (vpc-prod-style).

Without attaching it to the VPC, it cannot function.

Click on Action >> Select Attach to VPC >> Select VPC (created one)



Why Do We Need to Attach IGW to VPC?

If IGW is created but NOT attached:

- ✗ No internet access
- ✗ Public instances won't be reachable
- ✗ Route table will not work

IGW must be attached before routing traffic to it.

Traffic Flow until now

User → Internet → Internet Gateway (IGW) → Public Route Table → Public Subnet → EC2

Configuring Route Tables (Public & Private)

Route Tables control where traffic goes.

Even if IGW and NAT exist, without proper route configuration, nothing works.

Public Route Table Configuration

Create Public Route Table

Created a new Route Table

Attached it to my custom VPC

Associate Public Subnet

Opened **Subnet Associations**

Selected **Public Subnet**

Saved association

Now this route table controls the public subnet.

The screenshot displays the AWS Management Console interface for configuring a route table. The breadcrumb navigation shows the path: VPC > Route tables > rtb-089d65f8af1ede16e. The left-hand navigation pane is expanded to 'Virtual private cloud', with 'Route tables' selected. The main content area is titled 'rtb-089d65f8af1ede16e / public-rt-table' and includes an 'Actions' button. Below the title, there are four tabs: 'Routes' (selected), 'Subnet associations', 'Edge associations', and 'Route propagation'. The 'Routes' tab shows a table with two routes. The first route has a destination of '0.0.0.0/0', a target of 'igw-0614f29be38a83914', and a status of 'Active'. The second route has a destination of '10.0.0.0/16', a target of 'local', and a status of 'Active'. Above the routes table, there is a 'Details' section with information about the route table ID, VPC, and owner ID. There are also sections for 'Explicit subnet associations' and 'Edge associations'.

Details

Route table ID: rtb-089d65f8af1ede16e

VPC: vpc-0df286c3d919a89b5 | vpc-prod-style

Main: No

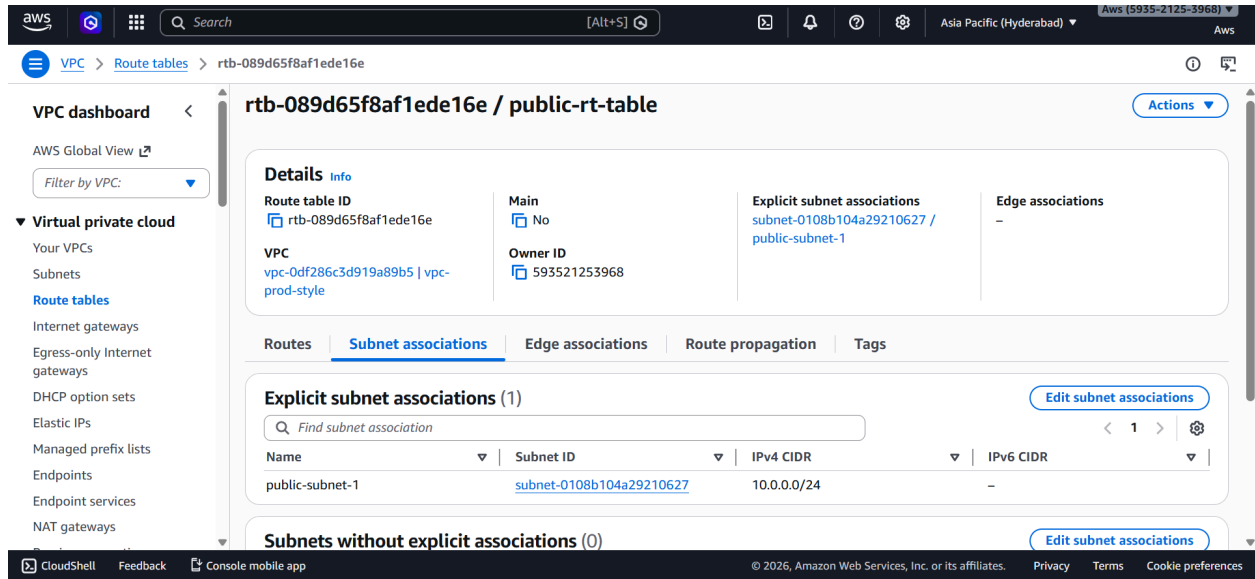
Owner ID: 593521253968

Explicit subnet associations: subnet-0108b104a29210627 / public-subnet-1

Edge associations: -

Routes (2)

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0614f29be38a83914	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table



Add Internet Route

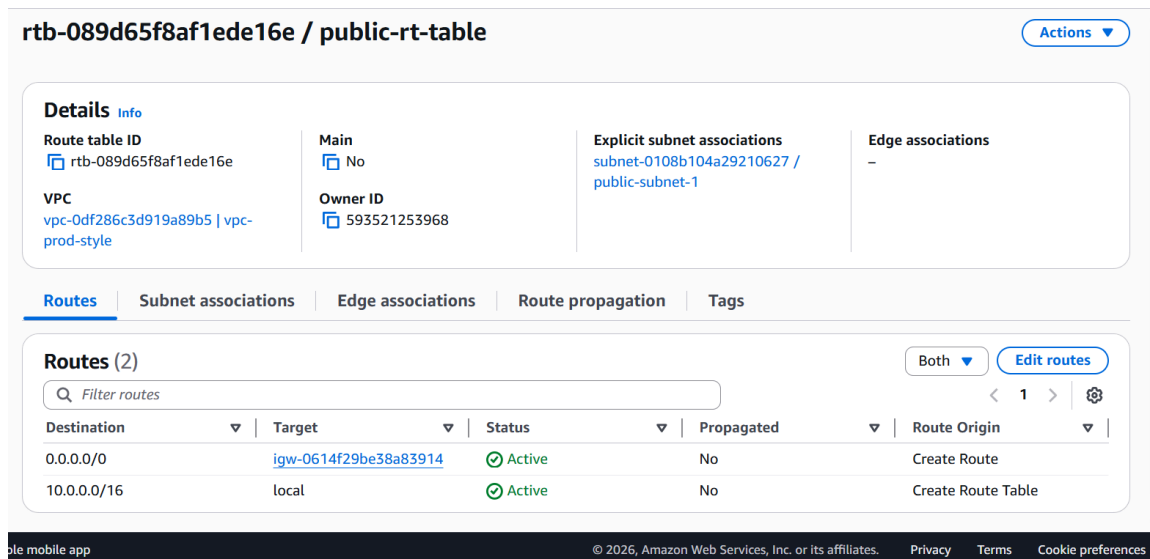
0.0.0.0/0 → Internet Gateway (IGW)

What Does This Mean:

0.0.0.0/0 = All external IP addresses

IGW = Send internet traffic to Internet Gateway

EC2 → Public Subnet → Public Route Table → IGW → Internet



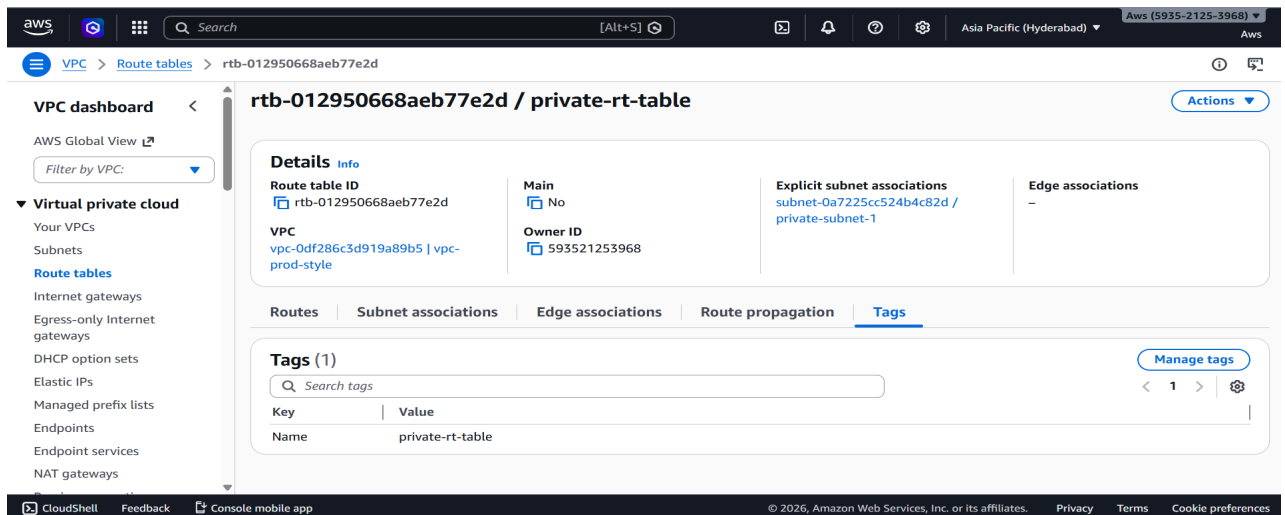
Inbound: User → Internet → IGW → Public Route Table → EC2

Private Route Table Configuration

Create Private Route Table

Created new Route Table

Attached to same VPC

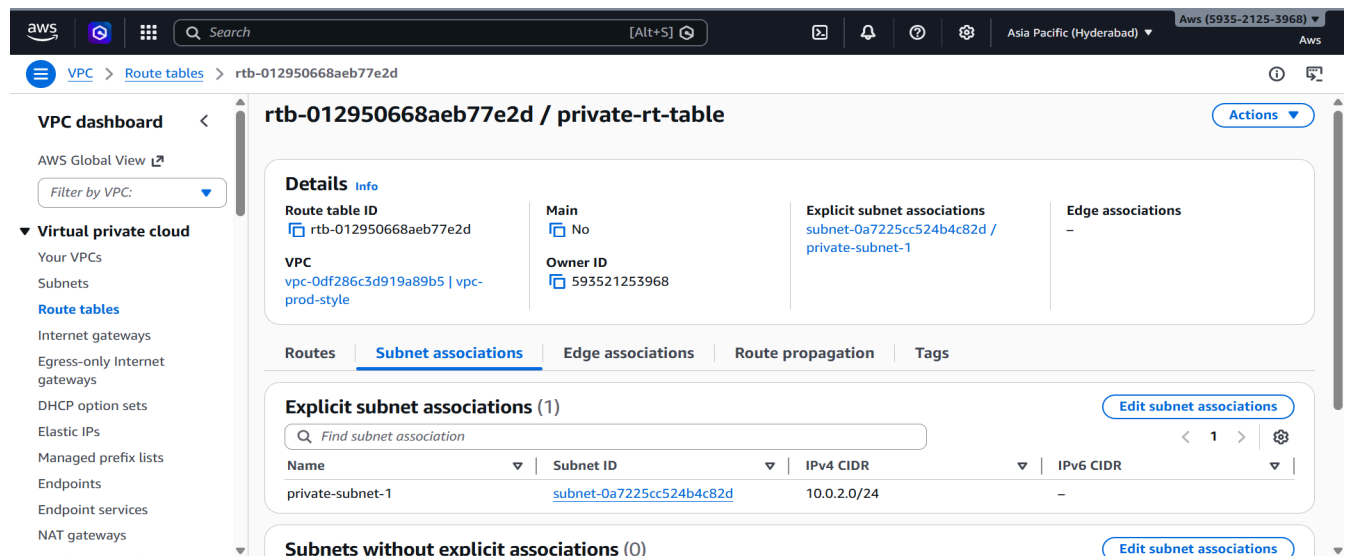


Associate Private Subnet

Selected Private Subnet

Saved association

Now this route table controls the private subnet.



Add NAT Route

Added this route:

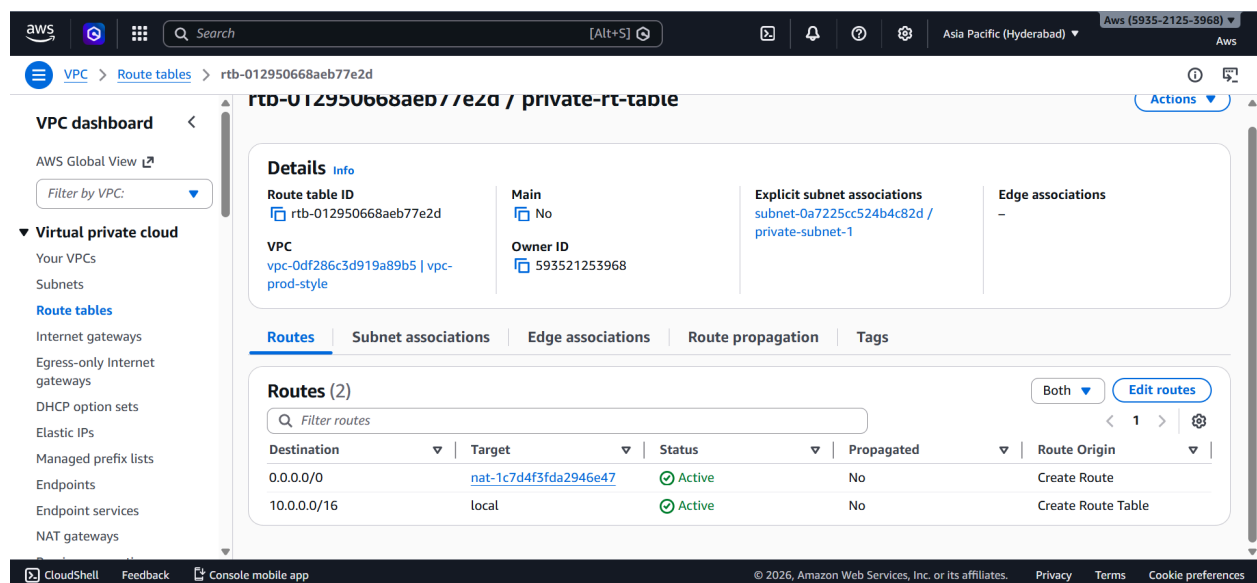
0.0.0.0/0 → NAT Gateway

What Does This Mean:

Send all internet-bound traffic to NAT
NAT will translate private IP to public IP

Flow:

Private EC2 → Private Route Table → NAT → IGW → Internet



Internet → IGW → NAT → Private EC2

Route Table is the brain of the subnet — it decides where traffic goes.

The public subnet brain says: “All outside traffic → Internet Gateway.”

The private subnet brain says: “All outside traffic → NAT (no direct internet).”

So Public = Open Gate | Private = Guarded Exit (outbound only).”

Creating NAT Gateway & Elastic IP

After configuring public routing, I needed to enable **secure internet access for private subnets**.

Private subnet should:

- ✓ Access internet (for updates, downloads)
- ✗ Not be directly accessible from internet

That's where **NAT Gateway** comes in.

Allocate Elastic IP

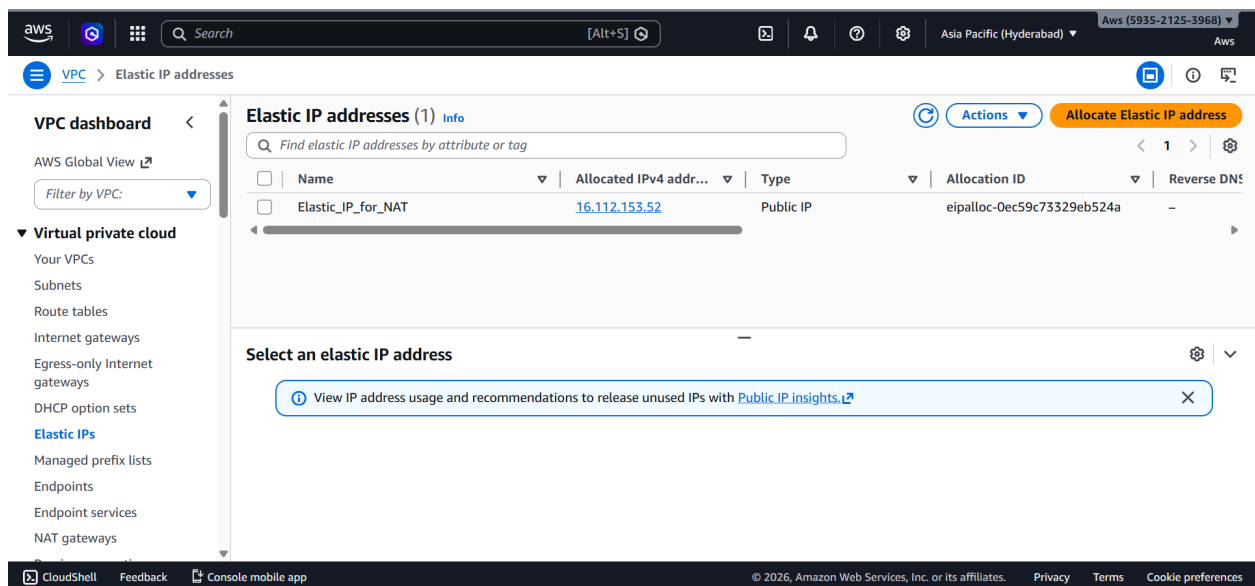
Allocated an Elastic IP

This provides a static public IP

Why :NAT Gateway requires a public IP to communicate with the internet.

Without Elastic IP:

- ✗ NAT cannot be created
- ✗ Private subnet cannot access internet



Create NAT Gateway (In Public Subnet)

Created NAT Gateway

Placed it inside **Public Subnet**

Attached allocated Elastic IP

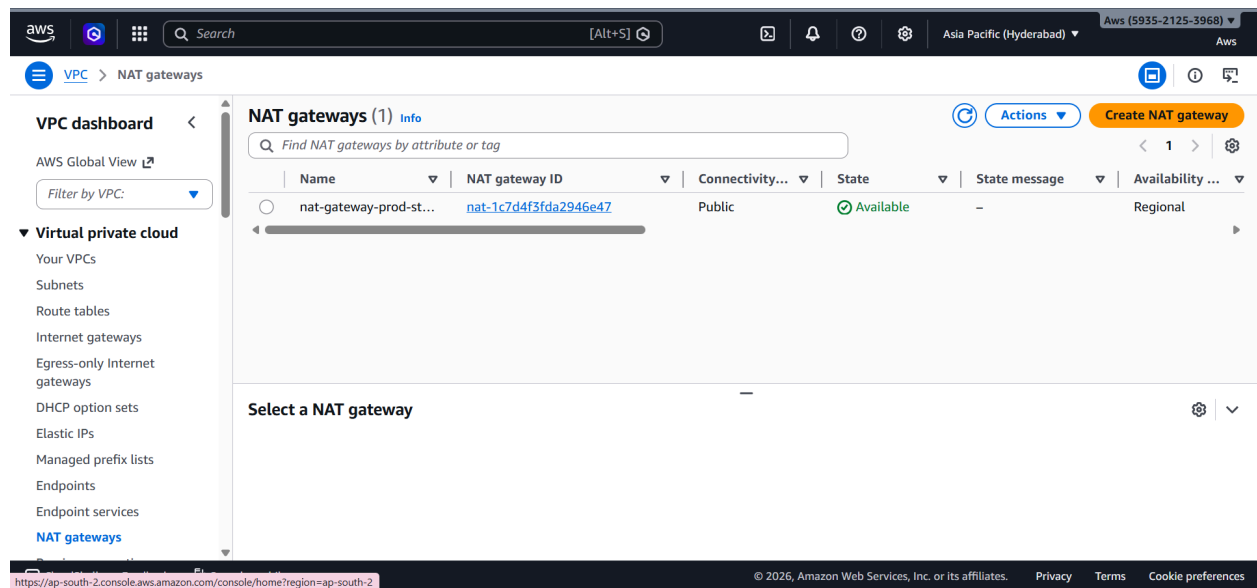
Why NAT Must Be in Public Subnet?

Because NAT needs:

- ✓ Route to Internet Gateway
- ✓ Public IP access

If placed in private subnet:

- ✗ It cannot reach IGW
- ✗ It becomes useless



Update Private Route Table

Added route: 0.0.0.0/0 → NAT Gateway

Now private subnets send internet traffic to NAT instead of IGW.

Added in private route table

Private Subnet Traffic Flow

Private EC2 → Private Route Table → NAT → IGW → Internet

VPC Resource Map Overview

After completing all configurations, I verified the entire architecture using the VPC Resource Map.

- VPC
- Public & Private Subnets
- Route Tables
- Internet Gateway
- NAT Gateway
- EC2 Instances

